

Truthful and Stable One-sided Matching on Networks

Tianyi Yang, Yuxiang Zhai, Dengji Zhao, Xinwei Song and Miao Li

ShanghaiTech University

{yangty2, zhaiyx, zhaodj, songxw, limiao}@shanghaitech.edu.cn

Abstract

Mechanism design on social networks is a hot research direction recently, and we have seen many interesting results in auctions and matching. Compared to the traditional settings, the new goal of the network settings is that we need to design incentives to incentivize the participants of the game to invite their neighbors on the network to join the game. This is challenging because they are competing for something (e.g., resources or matches) in the game. In one-sided matching, especially house exchange, the well-known unique truthful, stable and optimal solution called Top Trading Cycle (TTC) cannot achieve the new goal. Existing works have tried to add constraints on TTC to obtain the incentive, but it only works in trees and it does not guarantee any stability. In this paper, we move this forward and propose the first mechanism called Leave and Share (LS) which not only achieves the goal in all networks but also gives the most stable solution in the new settings. In terms of optimality, as it is impossible to achieve it in any network, we conduct simulations to compare it with the extensions of TTC.

1 Introduction

Incentivizing agents to invite new agents via their social connections is a new trend in mechanism design. Different from traditional static settings, the agents' connections are specifically considered and utilized by the mechanisms to enlarge the market [Zhao, 2021]. This is achieved by incentivizing the agents who are already in the game to invite their neighbors to join the game. In most games, a larger market contributes to a more desirable outcome. Particularly, in one-sided matching, more participants may lead to a more satisfied matching.

However, inviting more agents is not always better for the inviters. For example, A starts a matching game, and she invites her neighbors B and C to join in, and A prefers both B's and C's items. If in the end, B exchanges with C, and A does not exchange with any of them. However, if B is not in the game, A will exchange with C. In this example, A is not incentivized to invite B to this game.

To remove A's hesitation to invite all her neighbors, we should guarantee that the inviters' match is not getting worse after inviting their neighbors. In traditional one-sided matching, the well-known Top Trading Cycle (TTC) mechanism gives the unique truthful, stable and optimal solution [Shapley and Scarf, 1974; Ma, 1994]. However, TTC failed to incentivize the participants to invite others, because an invitee might compete with her inviters for the same match [Kawasaki *et al.*, 2021]. In order to achieve the incentive, we may add constraints on TTC such as each agent can only exchange with certain agents or the network has to be trees [Zheng *et al.*, 2020; Kawasaki *et al.*, 2021].

The restrictions on TTC indeed obtained the invitation incentive, but they limited the participants' matching choices and contradict the purpose of enlarging the game. The restrictions simply do not allow an invitee to have any chance to compete with her inviters, which also forbid a better allocation for those invitees who will not harm their inviters. Therefore, we still have space to further improve the matching by relaxing the restrictions. The challenge is that we cannot know whether an invitee will bring harm to her inviters or not without fixing a matching mechanism. That said, it seems that a natural way to find a better mechanism is through trial and error. However, the space and description of the matching mechanism are exponential in the number of participants, which is not doable in practice.

In this paper, we move this effort forward by proposing a new way of designing a better matching by relaxing a rather restricted mechanism. Our mechanism is based on a very restricted mechanism, called Swap With Neighbors (SWN), which only allows each agent to choose her preferred agent/item among her neighbors (also a restricted version of TTC). To relax the restriction, we allow matched agents to share their unmatched neighbors with others. By doing so, the later matched agents will have more choices, which is essential to improve their matching. We call this new mechanism Leave and Share (LS). This seems straightforward, but it is not because all agents would prefer matching later in order to get more shared choices from matched agents. Therefore, we need to carefully design the matching process to both utilize the shared benefits and also prevent new manipulations.

Finally, the LS mechanism can work for all networks (remove the structure restriction of the previous work). More

importantly, we also show that it gives the most stable solution under networks (which is also firstly proved here). In summary, our contributions advance the state of the art in the following ways:

- We prove the impossibilities of incentivizing invitation under the condition of optimality (stability). For stability, we extend the definition and prove what we can (cannot) achieve with invitation incentives.
- We propose the LS mechanism to achieve both invitation incentive and the extended stability for the first time. LS equals TTC if the network is a complete graph.
- To see the matching improvement under LS, we conduct simulations to compare LS with TTC extensions. The results indicate that the sharing feature of LS indeed improves the matching in expectation in all settings under consideration.

1.1 Related Work

Mechanism design over social networks is a hot research topic, where agents' social connections and interactions are considered by the mechanism. One popular way to utilize the connections is to attract new participants, which has made a significant progress in auctions and cooperative games [Li *et al.*, 2017; Li *et al.*, 2022; Zhang and Zhao, 2022]. In auctions, the main technique proposed to design the invitation incentives is to allow buyers to gain the social welfare increase due to their invitation. In cooperative games, invitees need to share their contributions with their inviters to get invited. Their methods rely on transferable utilities, which is not possible in matching.

Specifically, for one-sided matching over social networks, traditional solutions like TTC cannot be directly applied to incentivize participants' invitation. To make TTC work in the network setting, Kawasaki *et al.* [2021] presented a modified TTC under tree networks only. Their modification restricted each participant's choices to her parent and her subtree. Their extension incentivized invitation by disabling more satisfiable matchings. Gourvès *et al.* [2017] also studied one-sided matching in social networks, but they focused on static social connections and ignored agents' strategic behaviors. To evaluate the efficiency of a matching mechanism, some cardinal methods are well-studied in the traditional setting [Abraham *et al.*, 2005; Abebe *et al.*, 2020]. Motivated by these work, we also apply a cardinal method to evaluate our mechanism.

Besides standard one-sided matching, You *et al.* [2022] also studied a variant of house allocation problem [Abdulkadiroğlu and Sönmez, 1999] under the tree-structured networks. Moreover, Cho *et al.* [2022] investigated two-sided matching over social networks, where they presented a series of impossibilities to show the hardness of the problem, and designed mechanisms under the assumption that one side is known. The goal of these papers is to extend the traditional solutions to get the invitation incentives required in the corresponding network settings.

2 The Model

We consider a one-sided matching problem in a social network denoted by an undirected graph $G = (N, E)$, which

contains n agents $N = \{1, \dots, n\}$. Each agent $i \in N$ is endowed with an indivisible item h_i and $H = \{h_1, \dots, h_n\}$ is the set of all agents' items. We define agent i as j 's neighbor if there is an edge $e \in E$ between agent i and j , and let $r_i \subseteq N$ be i 's neighbor set.

Each agent $i \in N$ has a strict preference $\succ_i$ over H . $h \succ_i h'$ means i prefers h to h' and we use $\succeq_i$ to represent the weak preference. Denote agent i 's private type as $\theta_i = (\succ_i, r_i)$ and $\theta = (\theta_1, \dots, \theta_n)$ as the type profile of all agents. Let θ_{-i} be the type profile of all agents except for agent i , then θ can be written as (θ_i, θ_{-i}) . Let Θ be the type profile space of all agents. Similarly, we have $\Theta = (\Theta_i, \Theta_{-i})$.

In a matching mechanism, each agent is required to report her type (reporting neighbor set is treated as inviting neighbors in practice). We denote agent i 's reported type as $\theta'_i = (\succ'_i, r'_i)$, where $\succ'_i$ is the reported preference and $r'_i \subseteq r_i$ is the reported neighbor set. Let $\theta' = (\theta'_1, \dots, \theta'_n)$ be the reported type profile of all agents.

Definition 1. A one-sided matching mechanism is defined by an allocation policy $\pi = (\pi_i)_{i \in N}$, where $\pi_i : \Theta \rightarrow H$ satisfies for all $\theta \in \Theta$, for all i , $\pi_i(\theta) \in H$, and $\pi_i(\theta) \neq \pi_j(\theta)$ for all $i \neq j$.

Different from traditional settings, we assume only a subset of the agents are initially in the game (e.g., one agent initiated a matching with her neighbors). Without loss of generality, suppose an agent set $N_0 \subseteq N$ contains the initial participants in the matching. The others need the existing participants' invitation to join the game. As the invitation process is modeled by reporting their neighbors, we define the qualified participants by their reported types.

For a given report profile θ' , we generate a directed graph $G(\theta') = (N(\theta'), E(\theta'))$, where edge $\langle i, j \rangle \in E(\theta')$ if and only if $j \in r'_i$. Under θ' , we say agent i is qualified if and only if there is a path from any agent in N_0 to i in $G(\theta')$. That is, i can be properly invited by the invitation chain from agent set N_0 . Let $Q(\theta')$ be the set of all qualified agents under θ' . Then the matching mechanism can only use $Q(\theta')$.

Definition 2. A diffusion one-sided matching mechanism in social networks is a one-sided matching mechanism, $\pi = (\pi_i)_{i \in N}$, such that for all reported type profile θ' , it satisfies:

1. for all unqualified agents $i \notin Q(\theta')$, $\pi_i(\theta') = h_i$.
2. for all qualified agents $i \in Q(\theta')$, $\pi_i(\theta')$ is independent of the reports of all unqualified agents.

The difference between a diffusion one-sided matching and the matching defined in Definition 1 is that the participants can affect the qualification of other participants. If a participant changes her reported neighbor set, the qualified agent set may change. This is the challenge of this setting.

Next, we define two desirable properties for diffusion one-sided matching mechanisms: individual rationality and incentive compatibility. Intuitively, individual rationality requires that for each agent, reporting her type truthfully guarantees that she gets an item no worse than her own.

Definition 3 (Individual Rationality (IR)). A diffusion one-sided matching mechanism π is individually rational if for all $i \in N$, all $\theta_i \in \Theta_i$, and all $\theta'_{-i} \in \Theta_{-i}$, we have $\pi_i(\theta_i, \theta'_{-i}) \succeq_i h_i$.

	PO+IR	Stable	Stable-WCC	Stable-CC
IC	✗	✗	✗	✓

Table 1: The coexistence of IC with other properties.

For incentive compatibility, it means reporting type truthfully is a dominant strategy for each agent.

Definition 4 (Incentive Compatibility (IC)). *A diffusion one-sided matching mechanism π is incentive compatible if for all $i \in N$, all $\theta'_i \in \Theta_i$ and all $\theta_i, \theta'_i \in \Theta_i$, we have $\pi_i(\theta_i, \theta'_i) \succeq_i \pi_i(\theta'_i, \theta'_i)$.*

To evaluate the performance of a matching mechanism, an important metric is called Pareto optimality.

Definition 5 (Pareto Optimality (PO)). *A mechanism π is Pareto optimal if for all type profile θ , there is no other allocation $\pi'(\theta)$ such that for each agent i , $\pi'_i(\theta) \succeq_i \pi_i(\theta)$, and there exists at least one agent j , $\pi'_j(\theta) \succ_j \pi_j(\theta)$.*

Another metric is stability. A matching is stable if there does not exist any subset of agents who can deviate from the matching and match among the subset to make no one worse off, but at least one better off (this is called a blocking coalition). In the setting without networks, any subset of players can form a blocking coalition. However, in the network setting, they should know each other before they can form a coalition. Therefore, we assume that the blocking coalition in our setting is at least connected.

Definition 6 (Blocking Coalition). *Given an allocation $\pi(\theta)$, we say a set of agents $S \subseteq N$ (with item set $H_S \subseteq H$) is a blocking coalition for $\pi(\theta)$ if S forms a **connected component** in $G(\theta)$ and there exists an allocation $z(\theta)$ such that for all $i \in S$, $z_i(\theta) \in H_S$ and $z_i(\theta) \succeq_i \pi_i(\theta)$ with at least one $j \in S$ such that $z_j(\theta) \succ_j \pi_j(\theta)$.*

Definition 7 (Stability). *We say a mechanism π is stable if for all type profiles θ , there is no blocking coalition for $\pi(\theta)$.*

3 Impossibility Results

In this section, we discuss the impossibility results in the network setting presented in Table 1.

Theorem 1 (Impossibility for PO, IC and IR). *Given a social network with no less than three agents, no diffusion matching mechanism is PO, IC and IR.*

Proof. In the example shown in Figure 1, the only PO and IR allocations are a_3 and a_6 . For the former, agent 1 can misreport her preference as $h_3 \succ_1 h_1 \succ_1 h_2$. Under agent 1's misreport, the only PO and IR allocation will be a_6 , and 1 reaches a better allocation under a_6 thus violating IC. For the latter, agent 2 can misreport her neighbor set as $\{1\}$ and disqualify agent 3. In this way, the only PO and IR allocation is a_3 , 2 reaches a better allocation which also violates IC. Hence, no mechanism under the social network setting with no less than 3 agents is PO, IC and IR. $\square$

A similar result also holds even if the neighbor relationship is asymmetric[Kawasaki *et al.*, 2021].

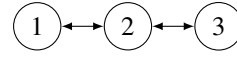


Figure 1: An example of social networks.

Preference	Allocation	PO+IR	TTC	Stable	SCC	SWCC
$h_3 \succ_1 h_2 \succ_1 h_1$	$a_1:(h_1, h_2, h_3)$	✗	✗	✗	✗	✗
	$a_2:(h_1, h_3, h_2)$	✗	✗	✗	✗	✗
$h_1 \succ_2 h_2 \succ_2 h_3$	$a_3:(h_2, h_1, h_3)$	✓	✗	✓	✓	✓
	$a_4:(h_2, h_3, h_1)$	✗	✗	✗	✗	✗
$h_1 \succ_3 h_3 \succ_3 h_2$	$a_5:(h_3, h_1, h_2)$	✗	✗	✗	✗	✗
	$a_6:(h_3, h_2, h_1)$	✓	✓	✓	✓	✓
$h_3 \succ'_1 h_1 \succ'_1 h_2$	$a_1:(h_1, h_2, h_3)$	✗	✗	✗	✓	✗
	$a_2:(h_1, h_3, h_2)$	✗	✗	✗	✗	✗
$h_1 \succ_2 h_2 \succ_2 h_3$	$a_3:(h_2, h_1, h_3)$	✗	✗	✗	✗	✗
	$a_4:(h_2, h_3, h_1)$	✗	✗	✗	✗	✗
$h_1 \succ_3 h_3 \succ_3 h_2$	$a_5:(h_3, h_1, h_2)$	✗	✗	✗	✗	✗
	$a_6:(h_3, h_2, h_1)$	✓	✓	✓	✓	✓

Table 2: All possible allocations of Figure 1 and whether an allocation satisfies specific properties. If agent 1 misreports, the allocation satisfies Stable-CC (SCC) will be a_1 and a_6 , instead of a_6 only as under stable or Stable-WCC (SWCC).

Theorem 2 (Impossibility for stability and IC). *Given a social network with no less than three agents, no diffusion matching mechanism is stability and IC.*

Proof. Consider the example given in Figure 1, the stable allocations are identical to the PO and IR ones (under the two type profile in Table 2). With the same strategic misreport, it can be concluded that stability is incompatible with IC. $\square$

To seek for an achievable stability in social networks, we should further restrict the blocking coalitions. In the traditional setting, since there are no constraints on social connections, the agents can be viewed as fully connected. Then, any blocking coalition is a complete component. Therefore, we require the blocking coalitions to be complete components.

Definition 8 (Blocking Coalition under Complete Components). *Given an allocation $\pi(\theta)$, we say a set of agents $S \subseteq N$ (with item set $H_S \subseteq H$) is a **blocking coalition under complete components** for $\pi(\theta)$ if S forms a complete component in $G(\theta)$ and there exists an allocation $z(\theta)$ such that for all $i \in S$, $z_i(\theta) \in H_S$ and $z_i(\theta) \succeq_i \pi_i(\theta)$ with at least one $j \in S$, $z_j(\theta) \succ_j \pi_j(\theta)$.*

Definition 9 (Stability under Complete Components (Stable-CC)). *We say a mechanism π is **stable under complete components** if for all type profiles θ , there is no blocking coalition under complete components for $\pi(\theta)$.*

In this paper, we will design a matching mechanism that satisfies IC, IR and Stable-CC. This extended stability looks rather restricted, is it possible to make a slight relaxation on the complete component? In fact, it is not achievable even if we just remove a single edge from a complete component.

Definition 10 (Nearly Complete Component). *We call a connected graph $G = (V, E)$ a **nearly complete component** if $|E| = \frac{|V| \cdot (|V| - 1)}{2} - 1$.*

The gap between a complete component and a nearly complete component is only one edge. We define a new stability

under nearly complete components and prove that it is impossible to coexist with IC.

Definition 11 (Blocking Coalition under Weakly Complete Components). *Given an allocation $\pi(\theta)$, we say a set of agents $S \subseteq N$ (with item set $H_S \subseteq H$) is a **blocking coalition under weakly complete components** for $\pi(\theta)$ if S forms a nearly complete component or a complete component in $G(\theta)$ and there exists an allocation $z(\theta)$ such that for all $i \in S$, $z_i(\theta) \in H_S$ and $z_i(\theta) \succeq_i \pi_i(\theta)$ with at least one $j \in S$, $z_j(\theta) \succ_j \pi_j(\theta)$.*

Definition 12 (Stability under Weakly Complete Components (Stable-WCC)). *We say a mechanism π is **stable under weakly complete components** if for all type profiles θ , there is no blocking coalition under weakly complete components for $\pi(\theta)$.*

Theorem 3 (Impossibility for Stable-WCC and IC). *Given a social network with no less than three agents, no diffusion matching mechanism is Stable-WCC and IC.*

Proof. Recall the example given in Figure 1, the possible blocking coalitions and allocations under Stable-WCC is identical to the example used in the proof of Theorem 2. Therefore, the same reasoning applies here. $\square$

4 The Mechanism

Before we introduce our mechanism, we first define the Top Trading Cycle and its extensions.

Definition 13 (Top Trading Cycle). *For a given $G(\theta')$, construct a directed graph by letting each agent point to the agent who has her favorite item remaining in the matching. There is at least one cycle. For each cycle, allocate the item to the agent who points to it and remove the cycle. Repeat the process until there is no agent left.*

TTC cannot ensure IC in the new setting, one trivial extension is called Swap With Neighbors (SWN), which only allows agents to swap with their neighbors. Intuitively, agents can only get allocated items from those who they invite and if they do not invite their neighbors, they will have no opportunity to get allocated items they prefer. Formal proof will be given in the appendix.

Definition 14 (Swap With Neighbors). *For a given $G(\theta')$, construct a directed graph by letting each agent point to her favorite item among herself and her neighbors remaining in the matching. There is at least one cycle. For each cycle, allocate the item to the agent who points to it and remove the cycle. Repeat the process until there is no agent left.*

Another attempt is to restrict the network to trees and allow each agent to swap with her neighbors and subtree [Kawasaki et al., 2021]. Let's call it Swap With Children (SWC).

Definition 15 (Swap With Children). *For a given $G(\theta')$, construct a directed graph by letting each agent points to her favorite item among herself, her neighbors, and her descendants remaining in the matching. There is at least one cycle. For each cycle, allocate the item to the agent who points to it and remove the cycle. Repeat the process until there is no agent left.*

Mechanism	Stable-CC	IC	
		Trees	All Networks
TTC	×	×	×
SWN	✓	✓	✓
SWC	✓(Trees)	✓	×
LS	✓	✓	✓

Table 3: Comparison on one-sided matching mechanisms over social networks.

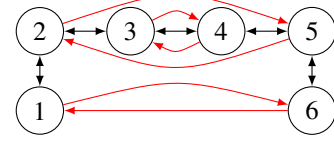


Figure 2: Preferences are $h_6 \succ_1 h_1 \succ_1 \dots$, $h_5 \succ_2 h_2 \succ_2 \dots$, $h_4 \succ_3 h_3 \succ_3 \dots$, $h_3 \succ_4 h_4 \succ_4 \dots$, $h_2 \succ_5 h_5 \succ_5 \dots$, $h_1 \succ_6 h_6 \succ_6 \dots$. One allocation for the agents 1 to 6 is $(h_6, h_5, h_4, h_3, h_2, h_1)$.

Both SWN and SWC avoid competition by restricting matching choices, which is not our goal to enlarge the market. We propose a new mechanism called Leave and Share (LS) which satisfies IC, IR and Stable-CC in all networks. Table 3 shows the difference of these mechanisms.

4.1 Leave and Share

Leave and Share uses SWN as a base and adds a natural sharing process to enlarge agents' selection space, trying to provide a better allocation. Firstly, agents are matched by rounds in a protocol that resembles SWN under a strategy-proof order. This guarantees that inviters are not worse off. Then, we share the neighbors of the left agents in this round by connecting their neighbors to each other, thus their neighbors can have new neighbors in the next round. This dynamic neighbor set update comes naturally because a matched cycle does not care how the remaining neighbors will be matched. Also, their remaining neighbors cannot prevent this sharing, and neither can the other remaining agents.

To see the value of our mechanism, consider the example given in Figure 2, where only agents 3 and 4 can exchange with each other in both SWN and SWC. The rest of the agents will end up with their own items. However, agents 3 and 4 will not block the exchange for agents 2 and 5 once they get their preferred items. After agents 3 and 4 are matched and **Leave**, we **Share** their remaining neighbors then agents 2 and 5 can swap. Similarly, after agents 2 and 5 leave, agents 1 and 6 can be matched as well. The process of Leave and Share is the name and core of our mechanism.

Before formalizing our mechanism, we introduce two notations to simplify the description.

Definition 16. *Given a set $A \subseteq N$, we say $f_i(A) = j \in A$ is i 's favorite agent in A if for any agent $k \in A$, $h_j \succeq_i h_k$.*

Definition 17. *An ordering of agents is a one-to-one function $\mathcal{P} : N^+ \rightarrow N$, where agent $\mathcal{P}(i)$ is the i^{th} agent in the ordering. Agents in $\mathcal{P}$ are sorted in ascending order by the length of the shortest path from agent set N_0 to them. Especially, for any agent $i \in N_0$, its shortest path length is 0. When multi-*

ple agents have the same length of the shortest path, we use a random tie-breaking.

Leave and Share (LS)

1. Initialize $N_{out} = \emptyset$ and an empty stack S . Define the top and bottom of S as S_{top} and S_{bottom} respectively, and let $R_i = r'_i \cup \{S_{bottom}, i\}$.
2. While $N_{out} \neq N$:
 - (a) Find the minimum t such that $\mathcal{P}(t) \notin N_{out}$. Push $\mathcal{P}(t)$ into S .
 - (b) While S is not empty:
 - i. While $f_{S_{top}}(R_{S_{top}}) \notin S$, push $f_{S_{top}}(R_{S_{top}})$ into S .
 - ii. Pop off all agents from S_{top} to $f_{S_{top}}(R_{S_{top}})$, who already formed a trading cycle C following their favorite agents. Allocate each agent $i \in C$ the item $h_{f_i(R_i)}$. Add C to N_{out}^t .
 - iii. Update the neighbor set of C 's remaining neighbors by removing C , i.e., for all $j \in \bigcup_{i \in C} r'_i \setminus N_{out}^t$, set $r'_j = r'_j \setminus C$.
 - (c) Add N_{out}^t to N_{out} . Let all remaining neighbors of N_{out}^t connect with each other, i.e., they become neighbors of each other. That is, let $X = \bigcup_{i \in N_{out}^t} r'_i \setminus N_{out}^t$ and for all $j \in X$, set $r'_j = r'_j \cup X$.

In LS, we first define an order $\mathcal{P}$ which depends on each agent's shortest distance to the initial agent set. Under this order, the first while loop (step 2) guarantees that the agent pushed into the stack is the remaining agent with the smallest order, and all agents are matched (including self-match) in the end. A new round begins each time the stack empties.

In the Leave stage, each agent that is pushed into the stack pushes her (current) favorite neighbor into the stack (step (a)). If her favorite agent is already in the stack, we pop all the agents between herself and her favorite agent to form a trading cycle. Specially, we allow the agent to choose the agent at the bottom of the stack as her favorite, which leads to the pop of all the agents in the stack (step (b)).

Once the stack is empty, the mechanism enters the Share stage and updates the neighbor set of the remaining agents (step (c)). All the neighbors of the left agents become new neighbors to each other. In the next Leave stage, they can choose their favorite neighbors in a larger neighbor set.

We illustrate how LS executes by an example. Consider the social network in Figure 3(a). The ordering is given as $\mathcal{P} = (1, 2, 3, 4, 5, 6, 7, 9, 8)$, $N_0 = 1$. The type profile θ is given in Table 4. The changes of N_{out}^t and r'_i in each turn are shown in Table 5. Figure 3(b) to Figure 3(d) show the process of Leave and Share, which runs as follows.

1. Starting at $\mathcal{P}(1)$, push $\mathcal{P}(1) = 1$ into the stack. $S_{top} = 1$, $f_1(R_1) = 2 \notin S$. Push agent 2 into the stack. $S_{top} = 2$, $f_2(R_2) = 4 \notin S$. Push agent 4 into the stack. $S_{top} = 4$, $f_4(R_4) = 5 \notin S$. Push agent 5 into

i	$\succ_i$	r_i
1	$h_2 \succ h_4 \succ h_3 \succ h_1 \succ \dots$	2
2	$h_4 \succ h_3 \succ h_5 \succ h_2 \succ \dots$	1,3,4
3	$h_8 \succ h_4 \succ h_1 \succ h_3 \succ \dots$	2,7
4	$h_5 \succ h_1 \succ h_3 \succ h_4 \succ \dots$	2,5,6,9
5	$h_2 \succ h_6 \succ h_9 \succ h_5 \succ \dots$	4,6,8
6	$h_4 \succ h_1 \succ h_8 \succ h_6 \succ \dots$	4,5
7	$h_8 \succ h_3 \succ h_6 \succ h_7 \succ \dots$	3
8	$h_9 \succ h_5 \succ h_1 \succ h_8 \succ \dots$	5
9	$h_7 \succ h_4 \succ h_6 \succ h_9 \succ \dots$	4

Table 4: The preference and neighbors of each agent. We omit those items which rank lower than their own items.

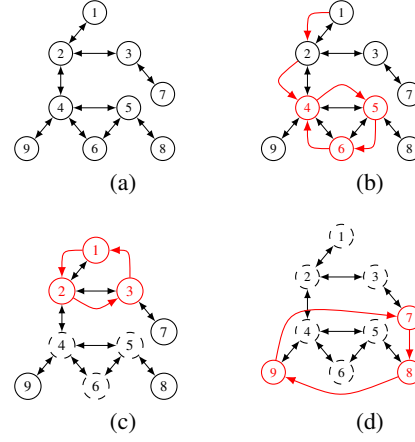


Figure 3: A running example of LS. Red nodes form the trading cycles, and red arrows point to their favorite neighbors. Dashed nodes denote agents who left the game.

the stack. $S_{top} = 5$, $f_5(R_5) = 6 \notin S$. Push agent 6 into the stack. $S_{top} = 6$, $f_6(R_6) = 4$, since $4 \in S$, pop off all agents from agent 6 to agent 4 to form the trading cycle $C = \{4, 5, 6\}$ and $N_{out}^1 = \{4, 5, 6\}$. Remove C and update $r'_2 = \{1, 3\}$, $r'_8 = \emptyset$, $r'_9 = \emptyset$.

2. Based on the new neighbor set, $S_{top} = 2$, $f_2(R_2) = 3 \notin S$. Push agent 3 into the stack. $S_{top} = 3$, $f_3(R_3) = 1$. Since $1 \in S$, pop off all agents from agent 3 to agent 1 to get $C = \{1, 2, 3\}$ and $N_{out}^1 = \{1, 2, 3, 4, 5, 6\}$. Remove C and update $r'_7 = \emptyset$.

Now that the stack is empty, update $N_{out} = N_{out}^1$ and connect the remaining neighbors of N_{out}^1 with each other, i.e., update $r'_7 = \{8, 9\}$, $r'_8 = \{7, 9\}$, $r'_9 = \{7, 8\}$.

3. Find new $t = 7$ and push $\mathcal{P}(7) = 7$ into the stack. $S_{top} = 7$, $f_7(R_7) = 8 \notin S$. Push agent 8 into the stack. $S_{top} = 8$, $f_8(R_8) = 9 \notin S$. Push agent 9 into the stack. $S_{top} = 9$, $f_9(R_9) = 7$. Since $7 \in S$, pop off all agents from agent 9 to agent 7 to get $C = \{7, 8, 9\}$ and $N_{out}^7 = \{7, 8, 9\}$.
4. The stack is empty again, add $N_{out}^7 = \{7, 8, 9\}$ to N_{out} , and now we have $N_{out} = N = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$. The mechanism terminates. The final allocation of the agents is shown in Table 5.

	t=1			t=7		Allocation
N_{out}^t	$\emptyset$	$\{4, 5, 6\}$	$\{1, 2, 3, 4, 5, 6\}$	$\emptyset$	$\{7, 8, 9\}$	$\pi_i(\theta)$
r_1^t	$\{2\}$	$\{2\}$	—	—	—	$\pi_1(\theta) = h_2$
r_2^t	$\{1, 3, 4\}$	$\{1, 3\}$	—	—	—	$\pi_2(\theta) = h_3$
r_3^t	$\{2, 7\}$	$\{2, 7\}$	—	—	—	$\pi_3(\theta) = h_1$
r_4^t	$\{2, 5, 6, 9\}$	—	—	—	—	$\pi_4(\theta) = h_5$
r_5^t	$\{4, 6, 8\}$	—	—	—	—	$\pi_5(\theta) = h_6$
r_6^t	$\{4, 5\}$	—	—	—	—	$\pi_6(\theta) = h_4$
r_7^t	$\{3\}$	$\{3\}$	$\emptyset$	$\{8, 9\}$	—	$\pi_7(\theta) = h_8$
r_8^t	$\{5\}$	$\emptyset$	$\emptyset$	$\{7, 9\}$	—	$\pi_8(\theta) = h_9$
r_9^t	$\{4\}$	$\emptyset$	$\emptyset$	$\{7, 8\}$	—	$\pi_9(\theta) = h_7$

Table 5: Updates of neighbor set for each agent. The allocation is shown in the rightmost column.

5 Properties of Leave and Share

In this section, we prove that LS is IR, IC and Stable-CC.

Theorem 4. *For any ordering $\mathcal{P}$, LS is IR.*

Proof. In LS, agent i leaves only when she gets an item h_j . Agent i can always choose herself as her favorite agent, then LS will allocate h_i to i . Thus, LS is IR. $\square$

Theorem 5. *For any ordering $\mathcal{P}$, LS is IC.*

Proof. Since each agent i 's type consists of two parts, her preference $\succ_i$ and her neighbor set r_i , we will prove misreporting neither $\succ_i$ nor r_i can improve her allocation.

Misreport on $\succ$: For agent i , we fix her reported neighbor set as r_i' . Her real preference is $\succ_i$ and reported preference is $\succ_i'$. Now we compare her allocation $\pi_i((\succ_i, r_i'), \theta_{-i}) = h_j$ with $\pi_i((\succ_i', r_i'), \theta_{-i}) = h_{j'}$.

Since $\mathcal{P}$ is based on the minimum distance, which is irrelevant to agents' preferences, we only need to prove that $h_j \succeq_i h_{j'}$ for all agents for a given order.

Before i is pushed into the stack, all trading cycles are irrelevant to $\succ_i'$ (i has not been preferred by the agents in the stack before, so $\succ_i'$ is not used at all). Thus we only consider the situation when agent i is pushed into the stack and then $\succ_i'$ can decide which agent after i is pushed into the stack.

When i is on the top of the stack, the next pushed agent $f_i(R_i)$ is determined by $\succ_i'$. Agent i can be allocated with $h_{j'}$ only when there is a trading cycle with i . Assume that $h_{j'} \succ_i h_j$, i.e., misreporting $\succ_i$ gives i a better item. We will show this leads to a contradiction. If i reported $\succ_i$ truthfully, then i would first choose j' before j (j' is pushed into the stack first), since i did not get $h_{j'}$, which means j' formed a cycle $C_{j'}$ without i . If reporting $\succ_i'$, i is matched with j' , then it must be the case that there exists another trading cycle B which breaks the cycle $C_{j'}$. Otherwise, whenever i points to j' , j' will form the original cycle $C_{j'}$ as it is independent of i 's preference. The only possibility for i to achieve this is by pointing her favorite agent under the false preference $\succ_i'$. By doing so, i can force other agents to leave earlier with different cycles including B . Next, we will show that it is impossible for B to break $C_{j'}$.

If B can actually break $C_{j'}$, there must be an overlap between B and $C_{j'}$. Assume that x is the node where B joins $C_{j'}$ and y is the node where B leaves $C_{j'}$ (x and y can be the same node). For node y , her match in B and $C_{j'}$ cannot be the same (the model assumes strict preference), and no matter when y is pushed into the stack, both items in B and $C_{j'}$ are

still there. Assume the matching in $C_{j'}$ is her favorite, then cycle B will never be formed. This contradicts to $h_{j'} \succ_i h_j$, so reporting $\succ_i$ truthfully is a dominant strategy.

Misreport on r : As the above showed for any reported neighbor set r_i' , reporting $\succ_i$ truthfully is a dominant strategy. Next, we further show that under truthful preference report, reporting r_i is a dominant strategy. That is, for the allocation $\pi_i((\succ_i, r_i), \theta_{-i}) = h_j$ and $\pi_i((\succ_i, r_i'), \theta_{-i}) = h_{j'}$, we will show $h_j \succeq_i h_{j'}$.

Firstly, we show that the tradings before i being pushed into the stack are irrelevant to i 's neighbor set report r_i' . For all the agents ranked before i in $\mathcal{P}$, their shortest distance is smaller than or equal to i 's shortest distance to agent 1, which means that their shortest paths do not contain i and therefore r_i' cannot change them. Thus, r_i' cannot change the order of all agents ordered before i in $\mathcal{P}$. In addition, agent i could be a cut point to disconnect certain agents D_i from agent 1, so r_i' can impact D_i 's distances and qualification. However, D_i can only be involved in the matching after i is in the stack, as others cannot reach D_i without i . Hence, before i is pushed into the stack, the tradings only depend on those ordered before i and the agents excluding D_i , which are independent of i . In fact, the order of the agents pushed into the stack before i is the same no matter what r_i' is. That is, when i is pushed into the stack, the agents, except for D_i , remaining in the game is independent of i .

Then when i misreports r_i , she will only reduce her own options in the favorite agent selection. Whether r_i' disconnects D_i or not, reporting r_i' here is equivalent to modifying $\succ_i$ by disliking neighbors in $r_i \setminus r_i'$. As we have showed, this is not beneficial for the agent. Therefore, reporting r_i truthfully is a dominant strategy, i.e., $h_j \succeq_i h_{j'}$.

Put the above two steps together, we have proved that LS is incentive compatible. $\square$

Theorem 6. *For any ordering $\mathcal{P}$, LS is Stable-CC.*

Proof. For every $S \subseteq N$ and their item set H_S . Let the allocation given by LS be $\pi(\theta)$. If there exists a blocking coalition S , where $S \subseteq N$ is the node set of a complete component in $G(\theta)$, we have $\forall i \in S, S \subseteq r_i$. A blocking coalition S suggests there exists a $z(\theta)$ such that for all $i \in S$, $z_i(\theta) \in H_S$, $z_i(\theta) \succeq_i \pi_i(\theta)$ with at least one $j \in S$ we have $z_j(\theta) \succ_j \pi_j(\theta)$. Therefore, for all $j \in S$, the blocking coalition guarantees the owner of $z_j(\theta)$ and j are in one trading cycle. This indicates if a trading cycle contains any agent in the coalition, all the agents in the trading cycle are in the coalition. Based on LS, $z_j(\theta) \succeq \pi_j(\theta)$ means the owner of $z_j(\theta)$ will be pushed into the stack before the owner of $\pi_j(\theta)$. Thus, the trading cycle which contains the owner of $z_j(\theta)$ and j can trade by the cycle (i.e., $\forall i \in S, z_i(\theta) = \pi_i(\theta)$). This contradicts the assumption of existing $j \in S, z_j(\theta) \succ_j \pi_j(\theta)$. Hence, LS is Stable-CC. $\square$

6 Optimality Analysis

In this section, we compare our mechanism with TTC, SWN and SWC. Since PO fails to be compatible with IC, IR in the network setting, we define a cardinal index D to measure the performance and run experiments in various random graphs

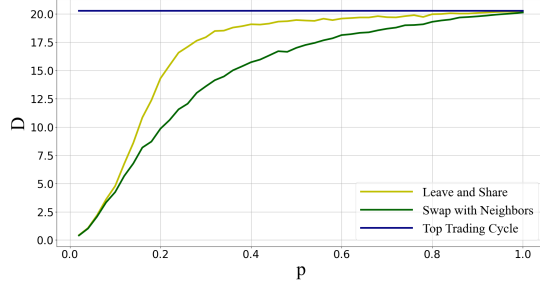


Figure 4: 100 graphs generated for each p to see how D changes accordingly. The minimum scale for p is 0.02.

to show the eminence of our mechanism. Although TTC cannot be directly applied in the social network setting, it provides an upper bound for the comparison. The lower bound is given by SWN since agents should be able to swap with their neighbors. We define $\succ_i(j)$ as the j^{th} favorite item of i . Assuming that h_i is $\succ_i(j)$ and $\pi_i(\theta)$ is $\succ_i(k)$, where $j \geq k$ for IR property, we define the ascension of i as $d_i = j - k$. The average ascension of agents is defined as $D = \frac{\sum_{i \in N} d_i}{n}$. We use D to measure the average improvement of agents' satisfaction in a one-sided matching mechanism.

Considering the lack of social network patterns in the random graphs, we adopt two special graphs, GIRG [Bringmann *et al.*, 2019] and small world [Watts and Strogatz, 1998]. Those graphs depict the cluster phenomenon in reality, and LS performs even better compared to that in the random graphs. Detailed results can be found in appendix B.

To generate random networks, we define the probability of an edge between any two nodes as p . A higher p leads to a denser connected graph. Especially, when $p = 1$, the graph is complete. To generate a tree, we use ub to represent the maximum number of child nodes for a tree, and for each node, we uniformly select an integer i from $[1, ub]$ as the number of the node's child nodes. Beginning at the root node, we create i child nodes for each node by a breadth-first order until the tree reaches size n . Agents' preferences are generated randomly from all permutations of the items.

Figure 4 shows the performances of three mechanisms. In this figure, we generate 100 graphs of 50 nodes with fixed but randomly generated preferences and adjust p to see how D changes. When p is close to 1, the performances of LS and SWN are close, and they are the same as TTC when $p = 1$. Due to the sharing process, LS converges to TTC faster than SWN. In the other extreme case, when p goes to 0, both LS and SWN have a poorer performance, because there are fewer neighbors to swap or share.

Next, we compare LS with SWC, the other extension of TTC only on trees, in two dimensions including (a) different tree sizes; (b) same tree size, but different tree structures.

In terms of tree size, we generate 100 different trees for each tree size. In Figure 5, LS outperforms SWC and SWN significantly as the incremental tree size. Also, the results showed that SWC and SWN are quite close, because the probability of forming a cycle with more than two agents is small

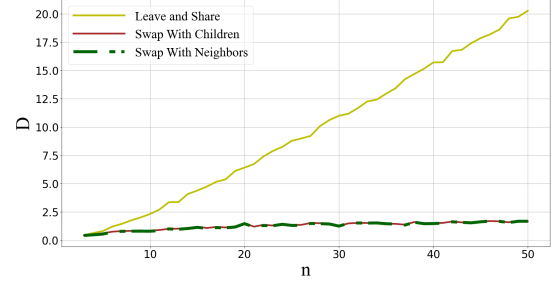


Figure 5: The tree is generated by a uniform distribution $[1, n - 1]$ for each node's child node number, n is the size of the tree. For LS, n and D are positively correlated.

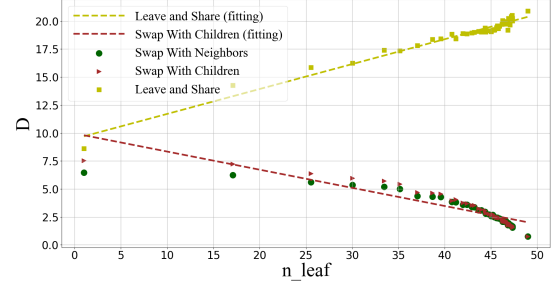


Figure 6: For LS, n_{leaf} and D are positive correlated. For SWN and SWC, n_{leaf} and D are negatively correlated.

when applying SWC. It requires all agents except for one in a cycle to prefer their parents' item rather than their whole subtrees'. The larger the cycle is, the smaller the probability is. Thus, by comparison to SWN, the improvement of SWC is very limited, while LS takes a big advantage from sharing.

As for tree structures, we fix the tree size and use the number of leaf nodes to indicate the difference between trees. For a certain tree with n nodes, the lower bound and upper bound of n_{leaf} are 1 and $n - 1$ respectively. We do the same simulation to show the relation between D and n_{leaf} .

In Figure 6, we generate 100 different trees for each ub from 1 to 49 and count the number of each tree's leaf nodes. Then we simulate three mechanisms on each tree. When n_{leaf} is small, SWC and LS are close, because there are few neighbors to share and form a big cycle. With the increase of n_{leaf} , LS performs better, because sharing can match nodes in different branches, even for leaf nodes. If allocated by SWC, those leaf nodes can only get their own items.

7 Conclusion

In this paper, we redefined stability in social networks and showed its tightness by proving impossibilities. We proposed a novel one-sided matching protocol called Leave and Share that satisfies IC, IR and Stable-CC. Our mechanism works in all networks and significantly outperforms other mechanisms. One possible future work is to find an attainable optimality in this new setting and design mechanisms to reach it.

A Properties for SWN

Theorem 7. *SWN is IR.*

Proof. In SWN, agent i can always choose herself as her favorite agent, then SWN will allocate h_i to i . Thus, SWN is IR. $\square$

Theorem 8. *SWN is IC.*

Proof. Since each agent i 's type consists of two parts, her preference $\succ_i$ and her neighbor set r_i , we will prove misreporting neither $\succ_i$ nor r_i can improve her allocation.

Misreport on $\succ$: For agent i , we fix her reported neighbor set as r'_i . Her real preference is $\succ_i$ and reported preference is $\succ'_i$. Now we suppose her allocation $\pi_i((\succ'_i, r'_i), \theta'_{-i}) = h_{j'}$ $\succ_i$ $\pi_i((\succ_i, r'_i), \theta'_{-i}) = h_j$.

In SWN, when i truthfully reports her preference, agent i is allocated h_j instead of $h_{j'}$, we know that $h_{j'}$ is in a trading cycle without i . Let the cycle containing j' be $C_{j'}$. When we fix the others preference and i misreports as $(\succ'_i, r'_i)$, since the trading cycle $C_{j'}$ is only determined by agents' preference in $C_{j'}$, which excludes agent i , $C_{j'}$ still forms. By SWN, agent i cannot be allocated $h_{j'}$, which contradicts our assumption.

Misreport on r : For agent i , we fix her reported preference as $\succ'_i$. Her real neighbor set is r_i and reported neighbor set is r'_i . Now we suppose her allocation $\pi_i((\succ'_i, r'_i), \theta'_{-i}) = h_{j'}$ $\succ'_i$ $\pi_i((\succ'_i, r_i), \theta'_{-i}) = h_j$.

In SWN, each agent can only be allocated the item from her reported neighbor set r'_i . Therefore, we have $j \in r_i$, $j' \in r_i$ and $j' \in r'_i$. Since with true neighbor set r_i , agent i is allocated h_j instead of $h_{j'}$, we know that $h_{j'}$ is in a trading cycle without i . Let the cycle containing j' be $C_{j'}$. According to SWN, each agent in $C_{j'}$ is pointing to her neighbor, which means their neighbor set is irrelevant to r_i as well as any r'_i . Therefore, the trading cycle $C_{j'}$ still remains and excludes agent i when agent i misreports r'_i . By SWN, agent i cannot be allocated $h_{j'}$, which contradicts our assumption.

Put the above two steps together, we have proved that SWN is incentive compatible. $\square$

Theorem 9. *SWN is Stable-CC.*

Proof. For every $S \subseteq N$ and their item set H_S . Let the allocation given by SWN be $\pi(\theta)$. If there exists a blocking coalition S , where $S \subseteq N$ is the node set of a complete component in $G(\theta)$.

Since S is the node set of a complete component, we have $\forall i \in S, S \subseteq r_i$. A blocking coalition S suggests there exists a $z(\theta)$ such that for all $i \in S$, $z_i(\theta) \in H_S$, $z_i(\theta) \succeq_i \pi_i(\theta)$ with at least one $j \in S$ we have $z_j(\theta) \succ_j \pi_j(\theta)$. Therefore, for all $j \in S$, the blocking coalition guarantees the owner of $z_j(\theta)$ and j are neighbors. Based on SWN, $z_j(\theta) \succ_j \pi_j(\theta)$ means j can always point to and get allocated $z_j(\theta)$ instead of $\pi_j(\theta)$. Thus, the trading cycle which contains the owner of $z_j(\theta)$ and j can trade by the cycle (i.e., $\forall i \in S, z_i(\theta) = \pi_i(\theta)$). This contradicts the assumption of existing at least one $j \in S \subseteq r_j, z_j(\theta) \succ_j \pi_j(\theta)$. Hence, SWN is Stable-CC. $\square$

B Additional Simulations

Due to the lack of real-world data sets, we apply two types of networks that capture the feature of social networks.

B.1 GIRG

GIRG first introduced by Bringmann *et al.* [2019] identifies the structural properties of social networks. It is a scale-free random network with large clustering coefficients. We carry out the same experiments regarding the average ascension in rank D as the increase of agent number n . Here we adopt the default parameter $\tau = 2.9, \alpha = 6$ in the GIRG model. We generate 100 random GIRG networks for 10 different n from 5 to 50.

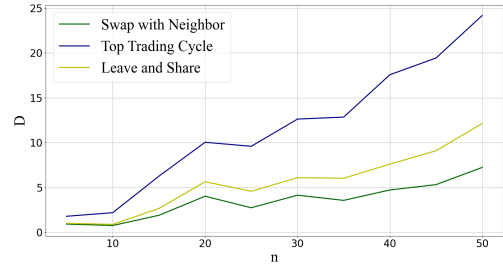


Figure 7: 100 GIRG graphs generated for each n to see how D changes accordingly.

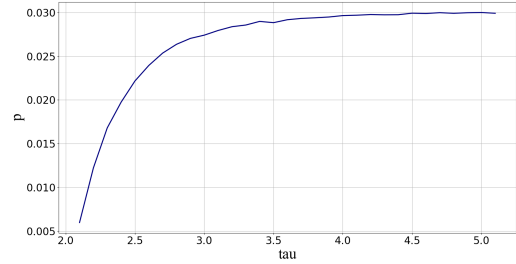


Figure 8: GIRG graph with $\alpha = 6$, see the change in expected connectivity with τ for 5000 times

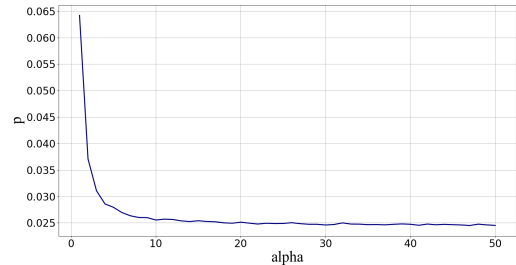


Figure 9: GIRG graph with $\tau = 2.9$, see the change in expected connectivity with α for 5000 times

As shown in Figure 7, LS outperforms SWN when the social network is a GIRG. Combining Figure 8 and Figure 9, we can see that GIRGS have a very narrow range of connectivity p . Also, comparing GIRGS to the randomly generated graph with same connectivity, LS’s performance in GIRGS is a lot better. One explanation is that GIRGs have many clusters, which makes the sharing process much more efficient.

B.2 Small World

Watts and Strogatz [1998] started the research regarding the small world phenomenon, and this particular network is widely used in simulations [Bakshy *et al.*, 2012; Phan and Airolidi, 2015; Goel *et al.*, 2009]. In their model, the parameter k indicates the expected neighbors number for each agent. We carry out the same experiments to observe the change in average ascension D and k . Note that, we can use $p = \frac{k}{n}$ to compute the expected connectivity. We generate 100 small world networks with 50 agents and a fixed preference for 10 different k from 5 to 50.

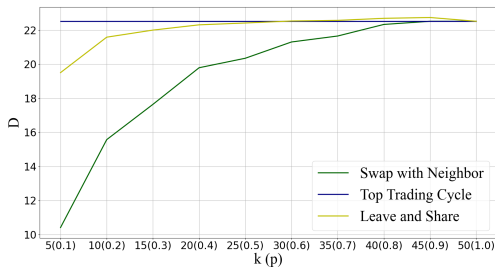


Figure 10: 100 small-world graphs generated for each k to see how D changes accordingly.

From Figure 10, we can see that LS performs much better in low connectivity compared with randomly generated graphs. Since the small world network model guarantees the graph to be a connected graph in extremely low connectivity, the sharing process of LS can always transform the network to be a complete graph. Therefore, the allocation given by LS is much closer to TTC.

References

- [Abdulkadiroğlu and Sönmez, 1999] Atila Abdulkadiroğlu and Tayfun Sönmez. House allocation with existing tenants. *Journal of Economic Theory*, 88(2):233–260, 1999.
- [Abebe *et al.*, 2020] Rediet Abebe, Richard Cole, Vasilis Gkatzelis, and Jason D. Hartline. A truthful cardinal mechanism for one-sided matching. In *31st Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2020*, pages 2096–2113, 2020.
- [Abraham *et al.*, 2005] David J. Abraham, Katarína Cechlárová, David F. Manlove, and Kurt Mehlhorn. Pareto optimality in house allocation problems. In *Algorithms and Computation*, pages 3–15, 2005.
- [Bakshy *et al.*, 2012] Eytan Bakshy, Dean Eckles, Rong Yan, and Itamar Rosenn. Social influence in social advertising: evidence from field experiments. In *Proceedings of the 13th ACM conference on electronic commerce*, pages 146–161, 2012.
- [Bringmann *et al.*, 2019] Karl Bringmann, Ralph Keusch, and Johannes Lengler. Geometric inhomogeneous random graphs. *Theoretical Computer Science*, 760:35–54, 2019.
- [Cho *et al.*, 2022] Sung-Ho Cho, Taiki Todo, and Makoto Yokoo. Two-sided matching over social networks. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, pages 186–193, 7 2022. Main Track.
- [Goel *et al.*, 2009] Sharad Goel, Roby Muhamad, and Duncan Watts. Social search in “small-world” experiments. In *Proceedings of the 18th international conference on World wide web*, pages 701–710, 2009.
- [Gourvès *et al.*, 2017] Laurent Gourvès, Julien Lesca, and Anaëlle Wilczynski. Object allocation via swaps along a social network. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 213–219, 2017.
- [Kawasaki *et al.*, 2021] Takehiro Kawasaki, Ryoji Wada, Taiki Todo, and Makoto Yokoo. Mechanism design for housing markets over social networks. In *AAMAS ’21: 20th International Conference on Autonomous Agents and Multiagent Systems, Virtual Event, United Kingdom, May 3-7, 2021*, pages 692–700, 2021.
- [Li *et al.*, 2017] Bin Li, Dong Hao, Dengji Zhao, and Tao Zhou. Mechanism design in social networks. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, February 4-9, 2017, San Francisco, California, USA*, pages 586–592, 2017.
- [Li *et al.*, 2022] Bin Li, Dong Hao, Hui Gao, and Dengji Zhao. Diffusion auction design. *Artificial Intelligence*, 303:103631, 2022.
- [Ma, 1994] Jinpeng Ma. Strategy-proofness and the strict core in a market with indivisibilities. *International Journal of Game Theory*, 23(1):75–83, 1994.
- [Phan and Airolidi, 2015] Tuan Q Phan and Edoardo M Airolidi. A natural experiment of social network formation and dynamics. *Proceedings of the National Academy of Sciences*, 112(21):6595–6600, 2015.
- [Shapley and Scarf, 1974] Lloyd Shapley and Herbert Scarf. On cores and indivisibility. *Journal of mathematical economics*, 1(1):23–37, 1974.
- [Watts and Strogatz, 1998] Duncan J Watts and Steven H Strogatz. Collective dynamics of ‘small-world’ networks. *nature*, 393(6684):440–442, 1998.
- [You *et al.*, 2022] Bo You, Ludwig Dierks, Taiki Todo, Mingming Li, and Makoto Yokoo. Strategy-proof house allocation with existing tenants over social networks. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems, AAMAS ’22*, page 1446–1454, 2022.

- [Zhang and Zhao, 2022] Yao Zhang and Dengji Zhao. Incentives to invite others to form larger coalitions. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, pages 1509–1517, 2022.
- [Zhao, 2021] Dengji Zhao. Mechanism design powered by social interactions. In *AAMAS '21: 20th International Conference on Autonomous Agents and Multiagent Systems, Virtual Event, United Kingdom, May 3-7, 2021*, pages 63–67, 2021.
- [Zheng *et al.*, 2020] Yue Zheng, Tianyi Yang, Wen Zhang, and Dengji Zhao. Barter exchange via friends’ friends. *arXiv preprint arXiv:2010.04933*, 2020.